PROJECT PORTFOLIO

Fashion-Tech AI Projects

WYA + Luna | Velaris | TryOn

Four full-stack applications that apply computer vision, deep learning and language models to fashion and jewellery: a wardrobe-intelligence platform on AWS, a conversational stylist built on top of it, an AI jewellery design tool that produces jeweller-ready specs and CAD, and a garment-fitting and 3D outfit builder. This document explains what each project does, how it is built, and how well it works.

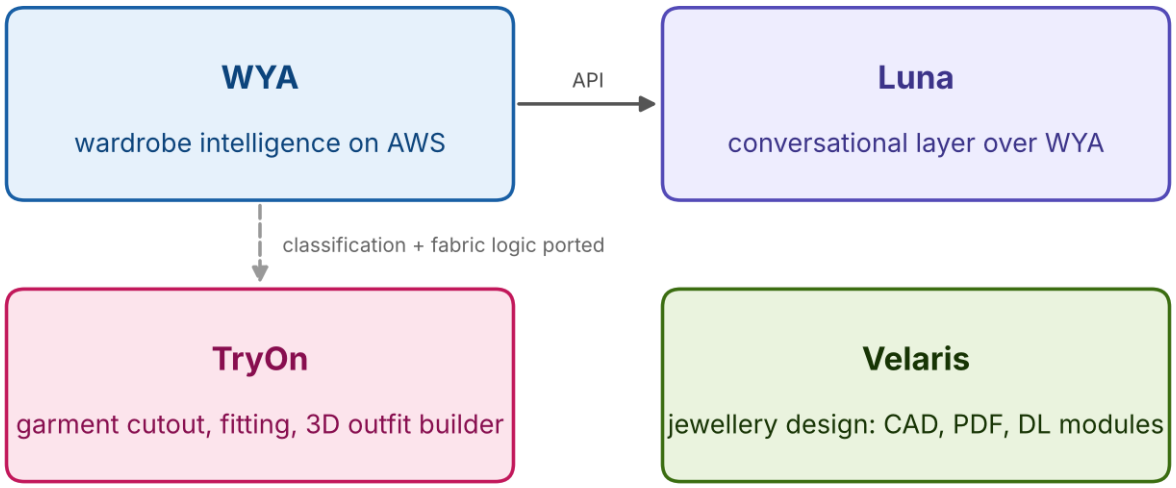


Figure 1. The four projects. Luna talks to WYA over its API; TryOn reuses WYA's classification and fabric logic; Velaris stands alone.

Projects at a glance

	WYA	Luna	Velaris	TryOn
Domain	Wardrobe + style intelligence	Conversational stylist	Jewellery design	Virtual outfit building
Input	Garment photos, style quiz	Natural-language messages	Text, sketch or photo	Garment screenshots / photos
Output	Tags, outfits, scores, aura card	Chat replies, outfit cards	Spec, cost, CAD SVG/DXF, PDF	Cutouts, fitted 2D/3D looks
AI used	FashionCLIP, KMeans, OpenCV rules	Intent rules + free-tier LLM chain	EfficientNet-B0, YOLOv8, CLIP, LLM	rembg, BlazePose, FashionCLIP, CatVTON
Backend	FastAPI + SQLite	FastAPI (proxy + router)	FastAPI + SQLite	FastAPI + SQLAlchemy/SQLite
Python (approx.)	8.7k lines	0.4k lines	5.5k lines	5.4k lines
Automated tests	pytest, 4 files	none in repo	pytest, 2 files	122 tests, 14 files
Status	Frontend live; AWS backend paused	Frontend live; servers paused	Runs locally	Runs locally

Author: Ria S | Document prepared November 2026 | Source: the project repositories (README files and code)

Table of Contents

WYA: What's Your Aesthetic . . . 3

- The problem . . . 3
- The solution . . . 3
- Screenshots . . . 3
- Features . . . 4
- System architecture . . . 4
- Garment auto-tagging pipeline . . . 4
- Engineering and operations . . . 5
- Future scope . . . 5

Luna: Conversational AI Stylist . . . 6

- The problem . . . 6
- What you can ask . . . 6
- How it works . . . 6
- A resilient LLM fallback . . . 7
- Screenshots . . . 7
- Deployment and engineering . . . 8

Velaris: From Inspiration to Jewellery Design . . . 9

- What it produces . . . 9
- Pipeline . . . 9
- Parametric CAD engine . . . 10
- Sample output: Phoenix Brooch . . . 10
- Deep-learning modules . . . 10
- Engineering . . . 11
- Known limitations . . . 11

TryOn: Build, Compare and Perfect Outfits . . . 12

- Core flow . . . 12
- How the fitting works . . . 12
- Evaluation: fitting vs baselines . . . 13
- Engineering and status . . . 14

How the projects fit together . . . 15

- Skills demonstrated . . . 15
- Current status and next steps . . . 15

WYA

What's Your Aesthetic: an AI-powered full-stack fashion web app

React + TypeScript | FastAPI | AWS SageMaker | GitHub Actions CI/CD

WYA: What's Your Aesthetic

Most people own more clothes than they wear and still feel they have nothing to wear. WYA turns a personal wardrobe into a queryable system: it reads garment photos with computer vision, builds a structured style profile, and uses both to generate outfits, sustainability scores and gap analysis. It was built with Mitakshi Sinha and is deployed on AWS.

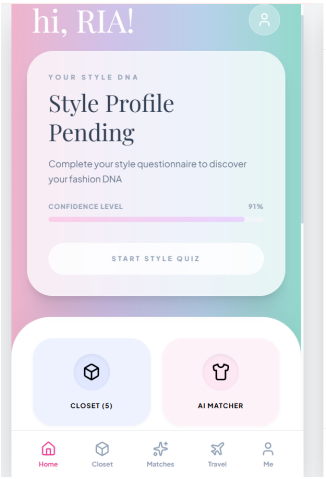
The problem

- **No visibility into what you own.** Wardrobes grow through impulse buys and gifts, so items are forgotten or re-bought.
- **Style is hard to articulate.** "Aesthetic" is vague, so shopping and curation are not actionable. Trend feeds are generic.
- **Sustainability is invisible.** There is no feedback loop between what people buy and its environmental cost.

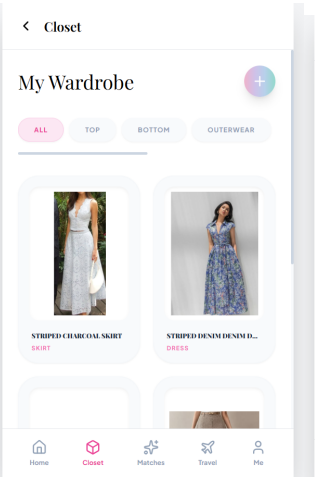
The solution

- **Style DNA Quiz** maps preferences (formality, colour palette, era, occasion) into a style vector that drives every recommendation.
- **AI wardrobe digitisation** extracts category, dominant and secondary colour, fabric, pattern, texture and brightness from a photo and writes a descriptive name such as "Floral Chiffon Midi Dress".
- **Outfit intelligence** builds combinations from clothes the user actually owns, using colour-harmony rules and the style profile.
- **Green Score** rates the wardrobe on brand scores, fabric composition and wear frequency, and shows which items drag it down.
- **Gap analysis** finds underserved categories or occasions so new purchases are intentional.

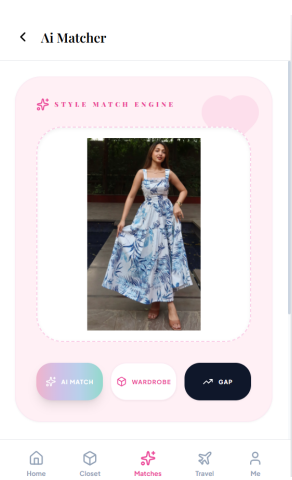
Screenshots



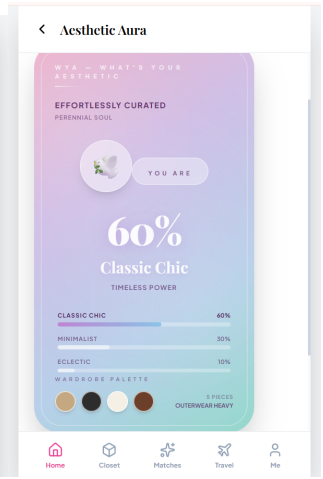
Home: Style DNA card and shortcuts



Closet: wardrobe grid with filters



AI Matcher: Style Match Engine



Aesthetic Aura: shareable style card

Features

Feature	What it does
Style DNA Quiz	Interactive questionnaire that maps a personal aesthetic to a profile vector
Wardrobe / Closet	Upload with AI auto-tagging (category, colour, fabric, pattern) and background removal via rembg
AI Outfit Matcher	Outfit suggestions from colour harmony (HSV colour theory) and the user's style profile
Style Evolution	Tracks how the aesthetic changes over time
Green Score	Sustainability rating from brand scores, fabric and wear frequency
Aesthetic Aura	Shareable "wrapped"-style card, e.g. Classic Chic 60%
Weather styling, Vacation Packer	Open-Meteo weather and Geoapify places drive outfit and packing suggestions
Push notifications	Style alerts and reminders using VAPID web push

System architecture

The React frontend is served from S3 behind CloudFront. CloudFront routes `/*` to S3 and `/api/*` to a Dockerised FastAPI backend on EC2, which keeps data in SQLite and calls a SageMaker endpoint for FashionCLIP. Routing the API through CloudFront gives end-to-end HTTPS with no mixed-content errors.

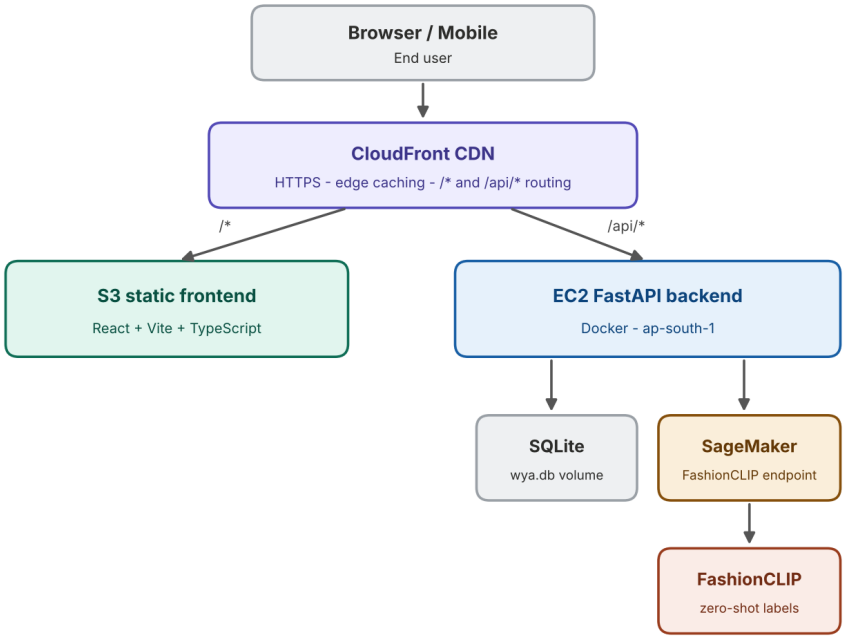


Figure 2. WYA deployment architecture (AWS, ap-south-1).

Garment auto-tagging pipeline

Tagging uses two tiers. **Tier 1** is zero-shot classification with FashionCLIP on SageMaker (the EC2 instance authenticates through an IAM instance profile, so no API keys are stored). **Tier 2** is a local rule-based fabric classifier that combines category, colour, texture and pattern. If SageMaker is unreachable the category falls back to "Top" and the rest of the pipeline still runs.

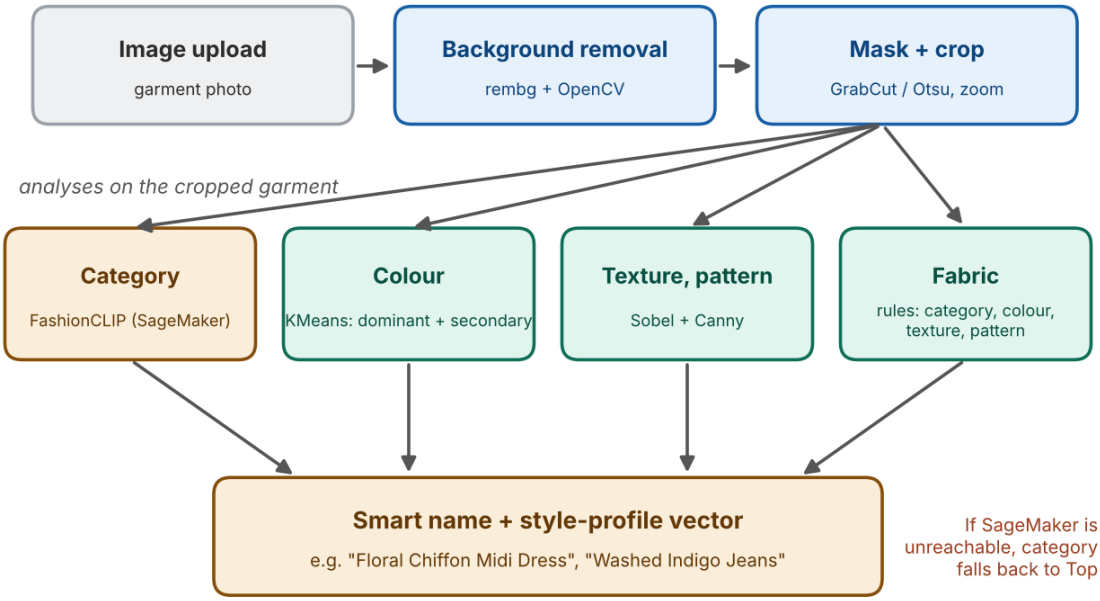


Figure 3. Image-to-tags pipeline: segmentation, parallel analyses, then name generation.

Engineering and operations

- **Backend:** FastAPI with about 55 routes across auth, wardrobe, AI, outfits, style, user, dashboard, feedback, health and a Luna bridge. JWT auth (python-jose, bcrypt).
- **Computer vision:** OpenCV (GrabCut, Otsu, Canny, Sobel, morphology), rembg, scikit-learn KMeans for colour, a custom colour-harmony engine and a rule-based fabric classifier.
- **Data layer:** JSON reference files for brand sustainability scores, colour dictionary and harmony, category map, regional items and weather codes.
- **Protection:** slowapi rate limits on AI endpoints (10 per minute for fabric-scan, outfit-match, curate-outfits and gap-analysis; 20 per minute for green-audit) to control SageMaker cost.
- **Reliability:** /health and /health/ready checks, a server watchdog, and automated daily S3 backups (these run when the EC2 instance is on).
- **CI/CD:** GitHub Actions on push to main: rebuild and restart the Docker backend, then build the frontend, sync to S3 and invalidate CloudFront.
- **Testing:** pytest suite (auth, wardrobe CRUD with cross-user isolation, health, outfits and rate-limit 429 checks) on a temporary SQLite database. The README cites 47 tests.
- **Status:** the frontend is live on CloudFront. The EC2 backend and SageMaker endpoint are stopped to pause AWS cost; a full demo video covers every feature.

Future scope

Outfit rating feedback loop, trend detection, virtual try-on, similar-item shopping, an outfit calendar, a React Native app and a barcode scanner. Deployment roadmap: AWS Secrets Manager and CloudWatch alerts, then Postgres on RDS with Redis and Celery, then autoscaling and multi-region only if real users arrive.

Luna

A conversational AI stylist: a chat interface that lets you talk to your wardrobe

React + Vite | FastAPI :8001 | Intent routing | OpenRouter LLM chain

Luna: Conversational AI Stylist

Wardrobe apps show what you own and fashion apps show what is trending, but neither lets you simply ask. Luna is a thin conversational layer over WYA. It holds no wardrobe data of its own: WYA is the brain and Luna is the mouth. Before the user says a word, Luna can already see their garments, aesthetic and gaps through WYA's API.

The problem

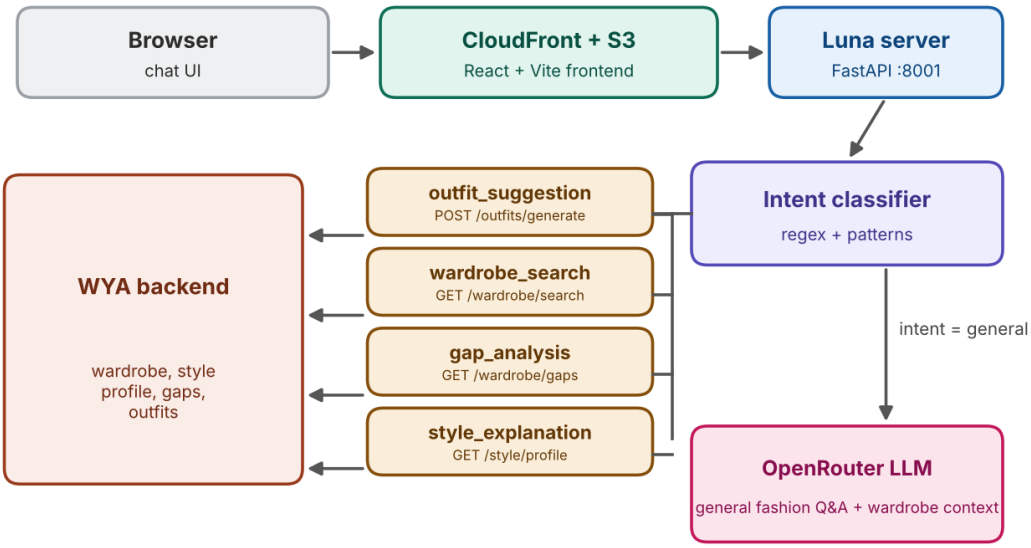
- **You cannot query your own closet.** There is no way to say "show everything navy" or "what do I have for a formal dinner?".
- **Style insights are buried in dashboards.** WYA builds a rich profile, but most users never navigate to it.
- **Outfit suggestions lack context.** Natural language can carry the occasion, the weather or the trip.

What you can ask

Message	What Luna does
"What should I wear to college tomorrow?"	Generates outfits from the real wardrobe
"Show all my black tops"	Natural-language search over garments
"What am I missing for winter?"	Gap analysis from WYA
"Why am I a minimalist?"	Explains the Style DNA in plain English
"Which colors go with burgundy?"	General fashion Q&A; with the wardrobe passed in as context

How it works

The browser sends a message to the Luna FastAPI server (three endpoints: `/health`, `/auth/login` and `/chat`). A regex intent classifier maps the text to one of five intents before any WYA call is made. Four intents route to a specific WYA endpoint; everything else goes to an LLM with the user's real garments included in the prompt. Login is proxied to WYA, so a WYA account and JWT are required.



Reply formatted and returned as a chat message

Figure 4. Luna request flow: intent classification, then a WYA route or the LLM fallback.

Intent	Example triggers	Handled by
outfit_suggestion	"what should I wear", "outfit for", "dress me"	POST /outfits/generate
wardrobe_search	"show me", "find my", "all my [colour/type]"	GET /wardrobe/search
gap_analysis	"what am I missing", "what do I need"	GET /wardrobe/gaps
style_explanation	"why am I", "what's my aesthetic"	GET /style/profile
general	anything else	OpenRouter LLM with wardrobe context

A resilient LLM fallback

Free-tier models get rate-limited or retired without notice, so the fallback tries an ordered list (gpt-oss-120b, Gemma 4 31B, Llama 3.3 70B). Each model gets one retry on a 429; a 404 skips straight to the next model. The system prompt forbids markdown and a regex cleanup pass acts as a backstop.

Screenshots

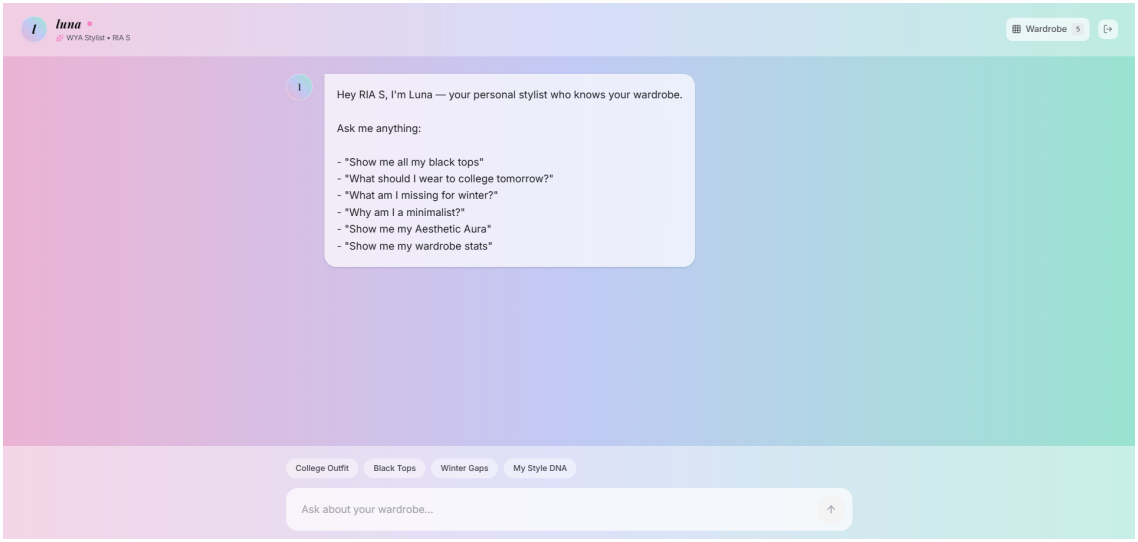


Figure 5. Luna chat screen: greeting, suggested prompts and quick-action chips.

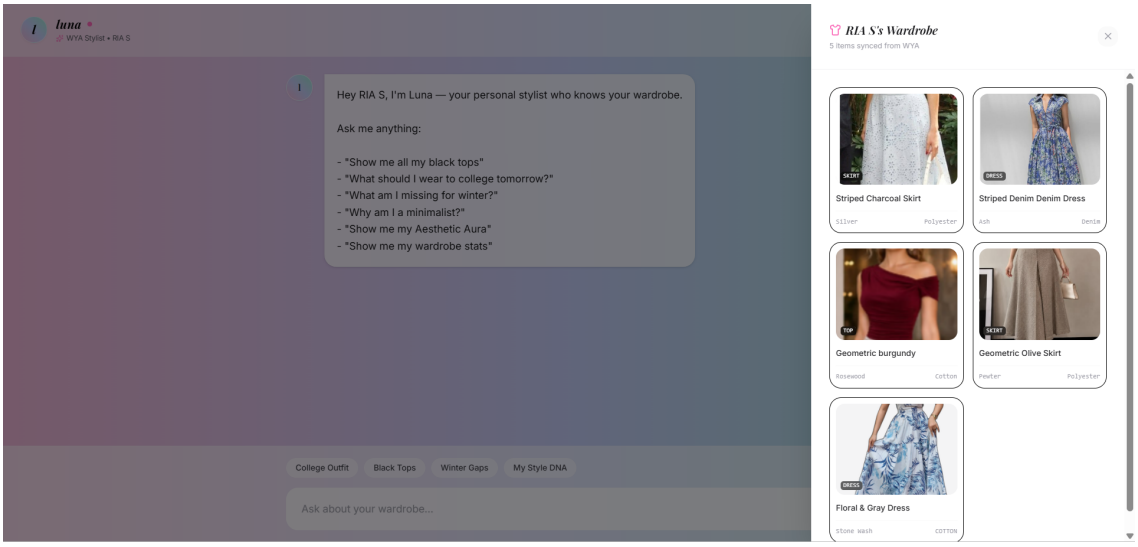


Figure 6. Wardrobe panel synced from WYA.

Deployment and engineering

- **Hosting:** React and Vite frontend on S3 and CloudFront; Luna server in Docker on the same EC2 instance as WYA (port 8001). CloudFront also proxies `/chat*` to the origin so the browser never makes an HTTP call from an HTTPS page.
- **CI/CD:** two parallel GitHub Actions jobs. The server job uses `git reset --hard origin/main` instead of a plain pull so local drift on EC2 cannot block or stale a deploy, then verifies the new container serves the deployed code.
- **Stack:** React, TypeScript, Vite, Tailwind; FastAPI with async httpx calls to WYA.
- **Status:** frontend live; WYA and Luna servers paused to manage AWS cost.

Velaris

From inspiration to jewellery design: type it, sketch it, upload it, get a design you can build

React 19 + TypeScript | FastAPI | 4 deep-learning modules | Parametric CAD + DXF

Velaris: From Inspiration to Jewellery Design

Custom jewellery is slow, expensive and prone to miscommunication. Customers struggle to describe what they want, see the result or understand cost, while jewellers spend hours interpreting vague briefs and making revisions before anything is made. Velaris turns a text description, a rough sketch or an inspiration photo into a design concept, a jeweller-ready specification, parametric CAD views and an exportable PDF.

What it produces

- A **type-accurate concept** for seven jewellery types (ring, necklace, pendant, earrings, bracelet, brooch, tiara), each with a distinct silhouette.
- A structured **specification sheet**: type, metal, stone, cut, setting, occasion, sizing and a cost breakdown (metal, stone, labour, markup).
- **Dynamic manufacturing notes** (casting, setting, polishing) driven by metal, setting type and a complexity score, so no two briefs read alike.
- A **presentation board** that combines CAD vectors with realistic renders, and a **PDF** with an attached true-scale **DXF**.
- **Persistent saved designs**, a gifting advisor and a trend-intelligence endpoint.

Pipeline

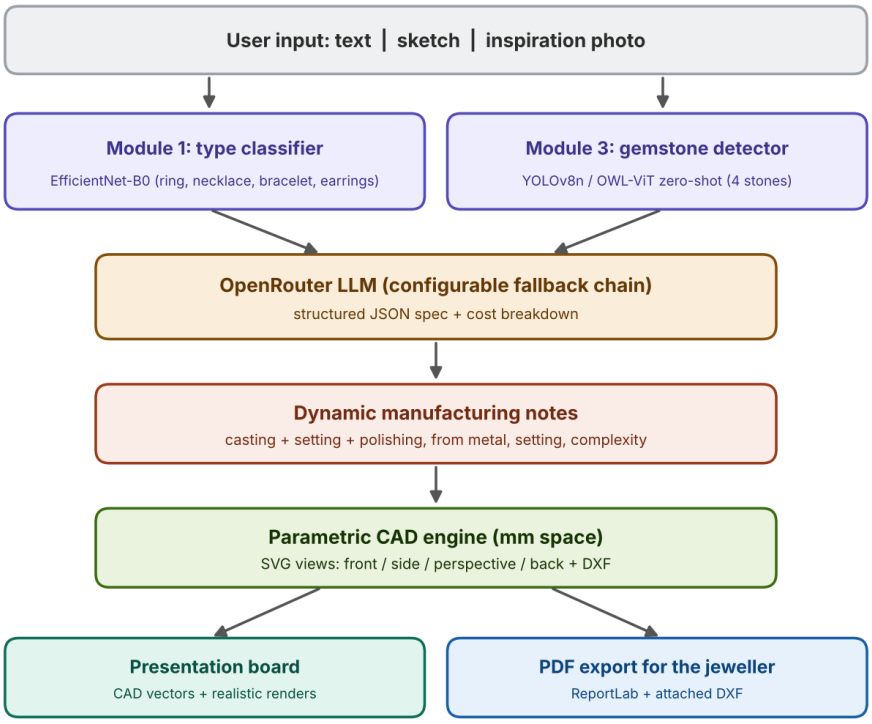


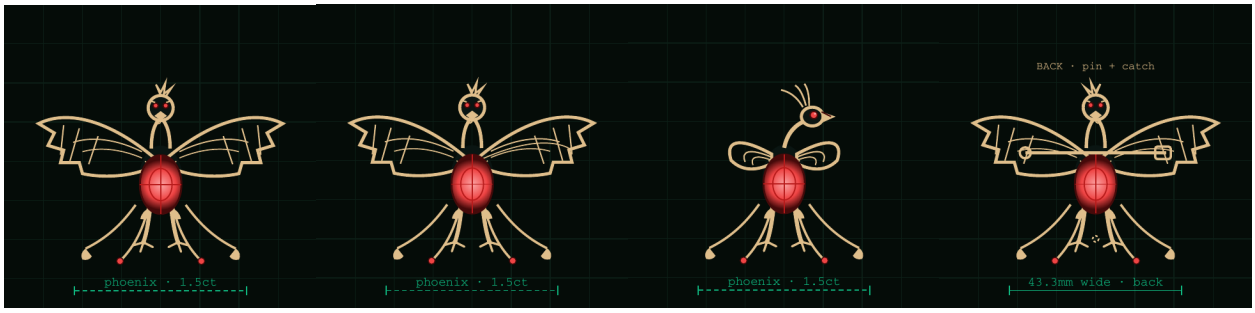
Figure 7. Velaris flow. Photo and sketch inputs pass through the type classifier and gemstone detector first; each step fails soft.

Parametric CAD engine

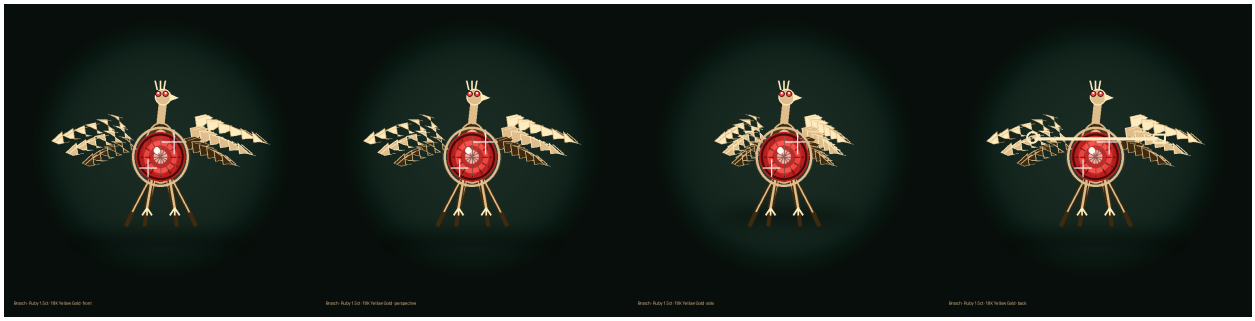
The visualiser is deterministic, with no LLM call on its primary path. A canonical `JewelryModel` in millimetre space is the single source of truth for the SVG views, the DXF export and the dimensions table, so front, side, perspective and back views stay geometrically consistent with each other and with the spec. Per-cut stone proportions (round, oval, pear, marquise, emerald, cushion, princess) and prong, bezel, halo and cathedral settings follow clearance rules, and invalid carat or dimension values are rejected with a 422. DXF files (R2010, mm units, layered BODY/STONE/SETTING/CONSTRUCTION/LABEL) are marked "CONCEPT - NOT FOR MANUFACTURE".

Sample output: Phoenix Brooch

The repository includes a PDF exported for a phoenix brooch. The CAD views below (top row) and procedural realistic previews (bottom row) come from that file.



CAD front | CAD perspective | CAD profile | CAD back (pin, catch)



Realistic front | Realistic perspective | Realistic side | Realistic back

Spec	Value	Cost component	Amount (USD)
Type / metal	Brooch, 18K yellow gold	Metal	$600
Stone	Ruby, oval, 1.5 ct, prong setting	Stone	$2,500
Overall size	43.3 x 49.4 x 8.0 mm	Labour	$500
Est. weight	5.9 g total (5.6 g metal)	Markup (25%)	$900
Complexity	68 / 100, moderate	Total retail	$4,500

Figures from the sample PDF dated September 20, 2026; dimensions and weights are estimates (see limitations).

Deep-learning modules

Four models sit beside the LLM flow. Each has an inference wrapper in `backend/ml/` and an offline training pipeline in `ml_training/`. Each activates when its checkpoint appears and fails soft otherwise, so the app runs unchanged without them.

Module	Model and task	Data and notes	Feeds design flow?
1. Type classifier	EfficientNet-B0, two-phase transfer learning: ring / necklace / bracelet / earrings	~6,100 images (sidd707 HF dataset); ring is the smallest class (~230). Falls back to keywords under 55% confidence	Yes

Module	Model and task	Data and notes	Feeds design flow?
2. Style classifier	EfficientNet-B0, binary: Traditional/Temple vs Modern/Minimal	Weak labels from caption keywords (distant supervision), not human-verified	No (API only)
3. Gemstone detector	YOLOv8n fine-tuned, or OWL-ViT zero-shot fallback: diamond, emerald, ruby, sapphire	Roboflow dataset; threshold 0.35 (YOLO) or 0.15 (OWL-ViT); result becomes an LLM hint	Yes
4. Similarity search	CLIP ViT-B/32 embeddings with cosine similarity, no training	Reference pool indexed once; returns top-k similar designs	No (API only)

Engineering

- **Stack:** React 19, TypeScript, Vite, Tailwind 4, motion; FastAPI with slowapi; SQLite in WAL mode; PyTorch, ultralytics, transformers; Pillow, OpenCV, ezdxf, ReportLab.
- **Model resilience:** OpenRouter fallback chain (overridable through `MODEL_FALLBACK_CHAIN`); diffusion "beauty" renders via Hugging Face FLUX fall back to a procedural PIL renderer and are disk-cached by spec hash.
- **Safety:** all backend-supplied SVG is sanitised with DOMPurify before rendering.
- **API surface:** about 20 endpoints covering design generation, save/list/delete, PDF export, CAD SVG/DXF, renders, presentation board, advisor, trends and the four ML modules.
- **Tests:** two pytest files with about 27 tests covering health, persistence, CAD and DXF round-trip, SVG sanitising, renders, board and PDF. Tests for live ML modules need locally trained models.

Known limitations

- CAD output is schematic, not production CAD. The DXF is a 2D front-view export, labelled not for manufacture. Dimensions and weights are heuristic estimates.
- Only one motif (phoenix) has a bespoke illustration. Module 1 covers four types; brooch, pendant and tiara rely on keyword matching.
- Modules 2 and 4 are not wired into the UI or design generation. The app runs locally only, with no Dockerfile yet.

TryOn

Build, compare and perfect outfits: upload it, cut it out, dress it up, see it for real

React Three Fiber | FastAPI | MediaPipe + rembg | Optional CatVTON

TryOn: Build, Compare and Perfect Outfits

Planning outfits from saved screenshots is clumsy: they pile up in a camera roll with no way to combine or compare them, and existing virtual try-on tools generate one slow, expensive photorealistic image at a time. TryOn separates **fast outfit composition** from **photorealistic rendering**. Any garment screenshot becomes a clean cutout, is fitted to a mannequin, and can be mixed into saved, comparable outfits. A photorealistic render is an optional extra that runs entirely on the user's machine.

Core flow

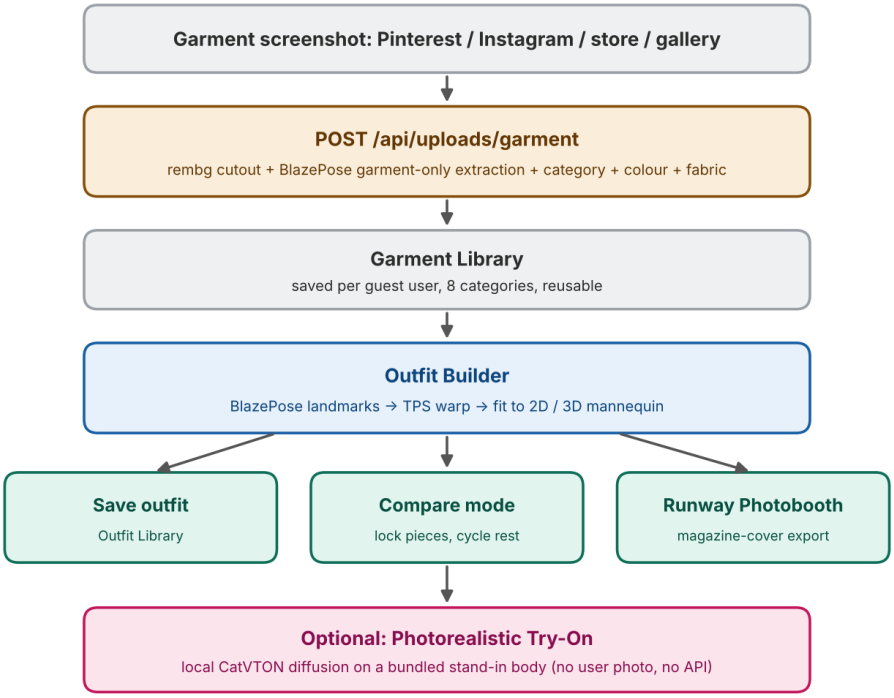


Figure 8. TryOn flow from screenshot to saved, compared and optionally photorealistic outfit.

How the fitting works

- **Cutout and extraction:** rembg removes the background; BlazePose-based garment-only extraction strips leftover head, limb and hand pixels. It records contours, bounding box and confidence, and deliberately never fills holes or invents fabric, so the saved asset contains only photographed garment pixels.
- **Auto-tagging:** FashionCLIP zero-shot category (decision tree ported from WYA), KMeans dominant colour mapped to a 58-name dictionary, and fabric from texture, pattern and rules (denim, silk, leather, knit, chiffon and others).
- **Fitting:** MediaPipe BlazePose landmarks and a thin-plate-spline (TPS) warp (OpenCV and SciPy), then silhouette conforming, edge feathering and depth shading, alpha-composited in category layer order.
- **3D mannequin:** React Three Fiber with feminine, masculine and neutral rigged GLTF models. Garments become a silhouette-fitted shell wrapping about 345 degrees around the body, so the back is covered when rotating.

- **Outfit tools:** a canvas for drag, resize, rotate, flip and lock; an Outfit Library with a dress/top/bottom exclusivity rule; Compare Mode; a style-score meter and XP quests; and a Runway Photobooth that exports a magazine-style card.
- **Photorealistic Try-On (optional):** local CatVTON diffusion on a bundled stand-in body, never a photo of the user. It needs one-time setup and GPU time (roughly 5 to 15 seconds on an 8 GB+ NVIDIA GPU, minutes on CPU). Weights are CC BY-NC-SA 4.0, so non-commercial.

Evaluation: fitting vs baselines

There is no labelled photo dataset in the repo, so the evaluation harness (`backend/evaluation/`) uses 10 procedurally generated garments (4 tops, 3 dresses, 3 bottoms). Every downstream stage runs on the real production code. The baselines are weaker compositions of the app's own warping functions: Resize, Affine, TPS, TPS plus silhouette, and the Proposed pipeline (similarity transform, silhouette conform, feathering, depth shading). Metrics are landmark error against the mannequin's reference body, silhouette IoU against its width profile, and a flare-distortion score.

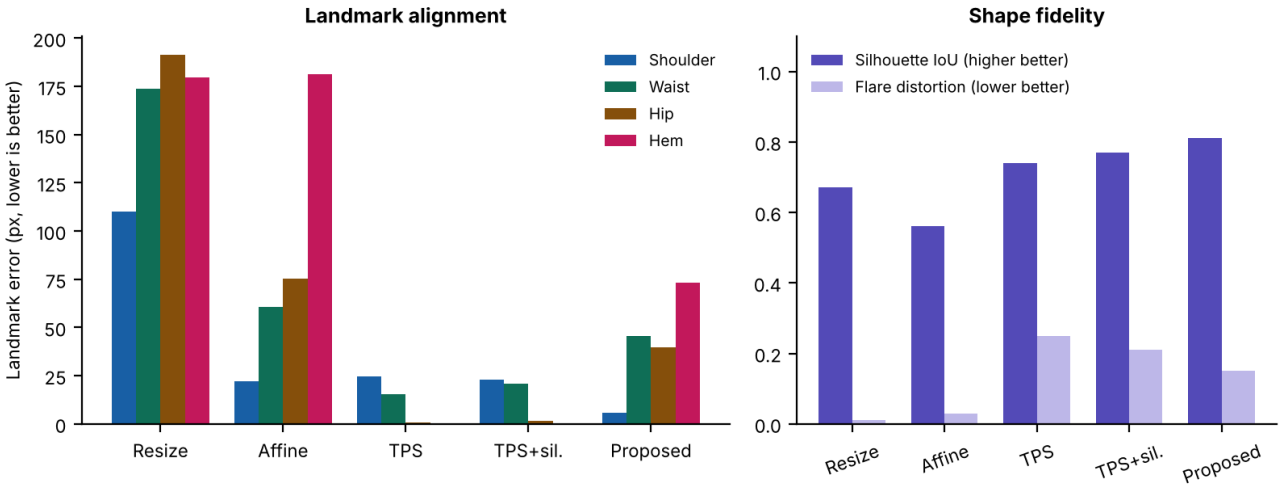


Figure 9. Baseline comparison on the synthetic benchmark (means; n differs by landmark, see results table).

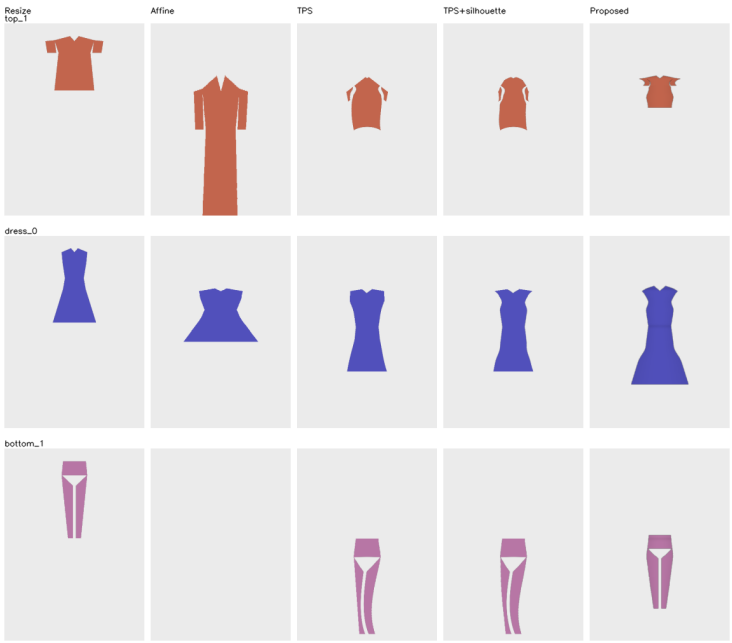


Figure 10. Qualitative grid for a top, a dress and trousers across the five methods (generated by run_experiments.py).

Reading the results honestly. The Proposed method has the best shoulder alignment (5.6 px versus 22 to 110 px for the others) and the best silhouette IoU (0.81). Plain TPS actually wins on hip and hem error (near 0 px) because it warps landmarks directly onto their targets, but it distorts the garment's own cut more (flare distortion 0.25 versus

0.15). The ablation shows the similarity transform matters most: removing it raises shoulder error from 5.6 to 101.4 px. This is a synthetic-geometry benchmark and does not show performance on real photos with occlusion, texture and lighting.

Engineering and status

- **Stack:** React 19, TypeScript, Vite, Tailwind 4, three.js; FastAPI, SQLAlchemy, SQLite, Pydantic; rembg, MediaPipe, FashionCLIP, OpenCV, SciPy.
- **Accounts:** per-browser guest accounts through an `X-User-Id` header, with 401 on a stale id and automatic re-registration.
- **Tests:** 122 backend tests in 14 files, including real rembg and MediaPipe running through the upload endpoint. The CatVTON path is not covered by automated tests.
- **Not yet done:** AWS S3 storage and CI/CD with cloud deployment. Cross-avatar 3D fit is wired but unverified in the browser, and very flared hems can show a faint seam at back-centre.

How the projects fit together

The four projects share one idea: use AI to make a visual, subjective craft (choosing clothes, designing jewellery) concrete enough to act on. They also share an engineering pattern: a React and TypeScript frontend, a FastAPI backend, pretrained vision models where they fit, and a deliberate fallback path so a missing model or rate-limited API degrades the result instead of breaking it.

Theme	WYA	Luna	Velaris	TryOn
Fallback design	SageMaker down: category Top	Ordered LLM chain with retry rules	ML modules fail soft; LLM chain; PIL renders	Heuristic classifier; optional CatVTON
Vision / ML	FashionCLIP, KMeans, OpenCV	Rule-based intent classifier	EfficientNet, YOLOv8, OWL-ViT, CLIP	rembg, BlazePose, FashionCLIP, diffusion
Quality evidence	pytest suite, rate-limit tests	Screenshots and live deploy	CAD/DXF round-trip tests	122 tests + baseline and ablation study
Deployment	AWS: S3, CloudFront, EC2, SageMaker	S3, CloudFront, EC2	Local	Local

Skills demonstrated

- **Applied ML and vision:** zero-shot classification, transfer learning, object detection, embedding search, segmentation, pose-based warping.
- **Full-stack engineering:** React and TypeScript, FastAPI, SQL storage, JWT and guest auth, rate limiting, 3D rendering with three.js.
- **Cloud and DevOps:** Docker, EC2, S3, CloudFront, SageMaker, IAM roles, GitHub Actions pipelines, health checks and backups.
- **Evaluation and honesty:** baseline and ablation studies, tests that run real models, and documented limitations in every README.

Current status and next steps

Project	State today	Most useful next step
WYA	Frontend live, backend paused for cost	Restart on demand; move to RDS Postgres and Secrets Manager when users arrive
Luna	Frontend live, servers paused	Add automated tests for the intent classifier and fallback chain
Velaris	Local only; ML modules need local training	Wire modules 2 and 4 into the UI; add Docker deployment
TryOn	Local only; evaluation on synthetic garments	Collect a labelled real-photo set; add S3 and CI/CD

Repositories: github.com/ria0304 (WYA-Whats-Your-Aesthetic, luna-stylist, Velaris-From-Inspiration-to-Jewellery-Design, TryOn). WYA is licensed PolyForm Strict 1.0.0 (noncommercial, no redistribution). CatVTON weights are CC BY-NC-SA 4.0.